



Imię i nazwisko	Wiktoria Sakowska (274931)
Nazwa uczelni	Uniwersytet Gdański
Nazwa wydziału	Matematyki, Fizyki i Informatyki
Kierunek	Informatyka Ogólnoakademicka
Prowadząca	mgr Laura Grzonka
Nazwa ćwiczenia	Projekt 4
Data oddania	10.06.2026 r.

HaluCheck

Aplikacja webowa do detekcji halucynacji AI

Dokumentacja projektu

Programowanie w języku Python — Projekt IV (Powiązanie projektów)

Spis treści

1	Opis projektu	3
2	Funkcjonalności	3
3	Technologie	4
4	Architektura aplikacji	4
4.1	Struktura projektu	4
4.2	Wzorce projektowe	5
4.3	Endpointy	5
5	Pipeline predykcji	5
6	Konsylium modeli	6
7	Przykładowe wyniki	7
7.1	Wykrycie halucynacji — Great Wall of China	7
7.2	Wykrycie halucynacji — Albert Einstein	8
7.3	Poprawna odpowiedź — Machine Learning	9
7.4	Poprawna odpowiedź — Human Heart	10
7.5	Wnioski z testów	11
8	Testy	11
9	Uruchomienie	11
10	Powiązanie z wcześniejszymi projektami	12
11	Podział pracy	12
11.1	Praca samodzielna	12
11.2	Z pomocą AI	12
12	Bibliografia	13
12.1	Sztuczna inteligencja	13
12.2	Dataset	13
12.3	Biblioteki i frameworki	13
12.4	Źródła internetowe	13

1 Opis projektu

Projekt HaluCheck łączy dwa wcześniejsze projekty z przedmiotu Programowanie w języku Python:

- **Project03** — klasyfikator halucynacji (6 modeli ML) wytrenowany na zbiorze HaluEval
- **Aplikacja webowa Flask** — interfejs umożliwiający użytkownikom sprawdzenie tekstu pod kątem halucynacji

Realizacja odpowiada przykładowi podanemu w specyfikacji projektu: „*Stworzenie strony do modelu ML*”.

Użytkownik wpisuje tekst wygenerowany przez AI (np. ChatGPT), a aplikacja:

1. Przetwarza tekst przez pipeline z Project03 (czyszczenie, TF-IDF, cechy numeryczne)
2. Uruchamia 6 modeli ML i wyświetla konsylium z głosowaniem większościowym
3. Pokazuje top słowa TF-IDF — na co model „patrzył” najbardziej
4. Analizuje cechy numeryczne tekstu i porównuje ze średnimi z datasetu HaluEval

2 Funkcjonalności

Funkcjonalność	Opis
Predykcja halucynacji	Analiza tekstu z użyciem modelu MLP jako głównego klasyfikatora
Konsylium 6 modeli	Wszystkie modele z Project03 głosują nad tekstem; wyświetlane są werdykty, pewność i prawdopodobieństwo halucynacji
Top słowa TF-IDF	15 słów z najwyższymi wagami TF-IDF — pokazuje cechy, na które model zwrócił uwagę
Analiza cech tekstu	10 cech numerycznych porównanych ze średnimi wartościami poprawnych i zhalucynowanych tekstów z HaluEval
Historia zapytań	Zapis ostatnich 20 zapytań w sesji przeglądarki z możliwością przeglądania i czyszczenia
Walidacja	Minimalna długość tekstu (10 znaków), komunikaty flash
Responsywność	Bootstrap 5, działanie na różnych urządzeniach

3 Technologie

Technologia	Wersja	Zastosowanie
Python	3.13	Język programowania
Flask	3.1.1	Framework webowy (app factory, blueprinty, szablony Jinja2)
scikit-learn	1.8.0	Modele ML (MLP, SVM, RF, LR, NB, XGBoost)
scipy	1.15.2	Macierze sparse (hstack, csr_matrix)
numpy	2.2.6	Obliczenia numeryczne
Bootstrap	5.3.3	Stylizacja interfejsu, responsywność
Bootstrap Icons	1.11.3	Ikony w interfejsie
pytest	8.3.5	Framework testowy (21 testów)

4 Architektura aplikacji

4.1 Struktura projektu

```
project04-halucheck/
|-- app/
|   |-- __init__.py           # app factory (create_app)
|   |-- routes.py             # endpointy: /, /predict, /history, /
|   |-- about
|   |-- model_loader.py       # ładowanie modeli, preprocessing,
|   |-- predykcja
|   |-- templates/
|       |-- base.html         # szablon bazowy z navbar i footer
|       |-- index.html        # formularz do wpisania tekstu
|       |-- result.html       # wynik predykcji z 3 panelami
|       |-- history.html      # tabela historii zapytan
|       |-- about.html        # opis projektu i modelu
|   |-- static/
|       |-- css/
|           |-- style.css     # dodatkowe style
|-- model/
|   |-- mlp.pkl               # MLP (Multi-Layer Perceptron)
|   |-- logistic_regression.pkl
|   |-- naive_bayes.pkl
|   |-- linear_svm.pkl
|   |-- random_forest.pkl
|   |-- xgboost.pkl
|   |-- tfidf_vectorizer.pkl # dopasowany TfidfVectorizer
```

```

|-- tests/
|   |-- __init__.py
|   '-- test_app.py           # 21 testow pytest
|-- run.py                   # punkt wejścia
|-- requirements.txt
|-- LICENSE
'-- README.md

```

4.2 Wzorce projektowe

- **App Factory** — funkcja `create_app()` tworzy i konfiguruje instancję Flask, umożliwiając łatwe testowanie i konfigurację
- **Blueprint** — endpointy zorganizowane w Blueprint `main`, oddzielony od app factory
- **Lazy Loading / Cache** — modele ML ładowane są raz (przy pierwszym użyciu) i przechowywane w zmiennych globalnych modułu, co eliminuje powtarzane odczytywanie plików `.pkl`
- **Separacja odpowiedzialności** — `model_loader.py` odpowiada za ML, `routes.py` za logikę HTTP, szablony za prezentację

4.3 Endpointy

Metoda	URL	Funkcja	Opis
GET	/	<code>index</code>	Strona główna z formularzem
POST	<code>/predict</code>	<code>predict_route</code>	Predykcja halucynacji
GET	<code>/history</code>	<code>history</code>	Historia zapytań
POST	<code>/history/clear</code>	<code>clear_history</code>	Czyszczenie historii
GET	<code>/about</code>	<code>about</code>	Informacje o projekcie

5 Pipeline predykcji

Pipeline predykcji odtwarza dokładnie ten sam proces, który był stosowany przy treningu modeli w Project03:

1. **Ekstrakcja cech numerycznych** (z oryginalnego tekstu, przed czyszczeniem) — 10 cech: długość tekstu, liczba słów, średnia długość słowa, liczba zdań, wykrzykniki, znaki zapytania, udział wielkich liter, udział unikalnych słów, liczba cyfr, udział interpunkcji
2. **Czyszczenie tekstu** — normalizacja białych znaków, konwersja do lowercase
3. **Transformacja TF-IDF** — użycie tego samego `TfidfVectorizer` (10 000 cech, uniagramy + bigramy) co przy treningu
4. **Łączenie cech** — `hstack` macierzy TF-IDF (10 000 cech) z cechami numerycznymi (10 cech) = 10 010 cech
5. **Predykcja** — model zwraca label (0/1) i prawdopodobieństwa klas

Kluczowe jest użycie **tego samego vectorizera** (`tfidf_vectorizer.pkl`), który był dopasowany na danych treningowych — nowy tekst musi być transformowany do identycznej przestrzeni cech.

6 Konsylium modeli

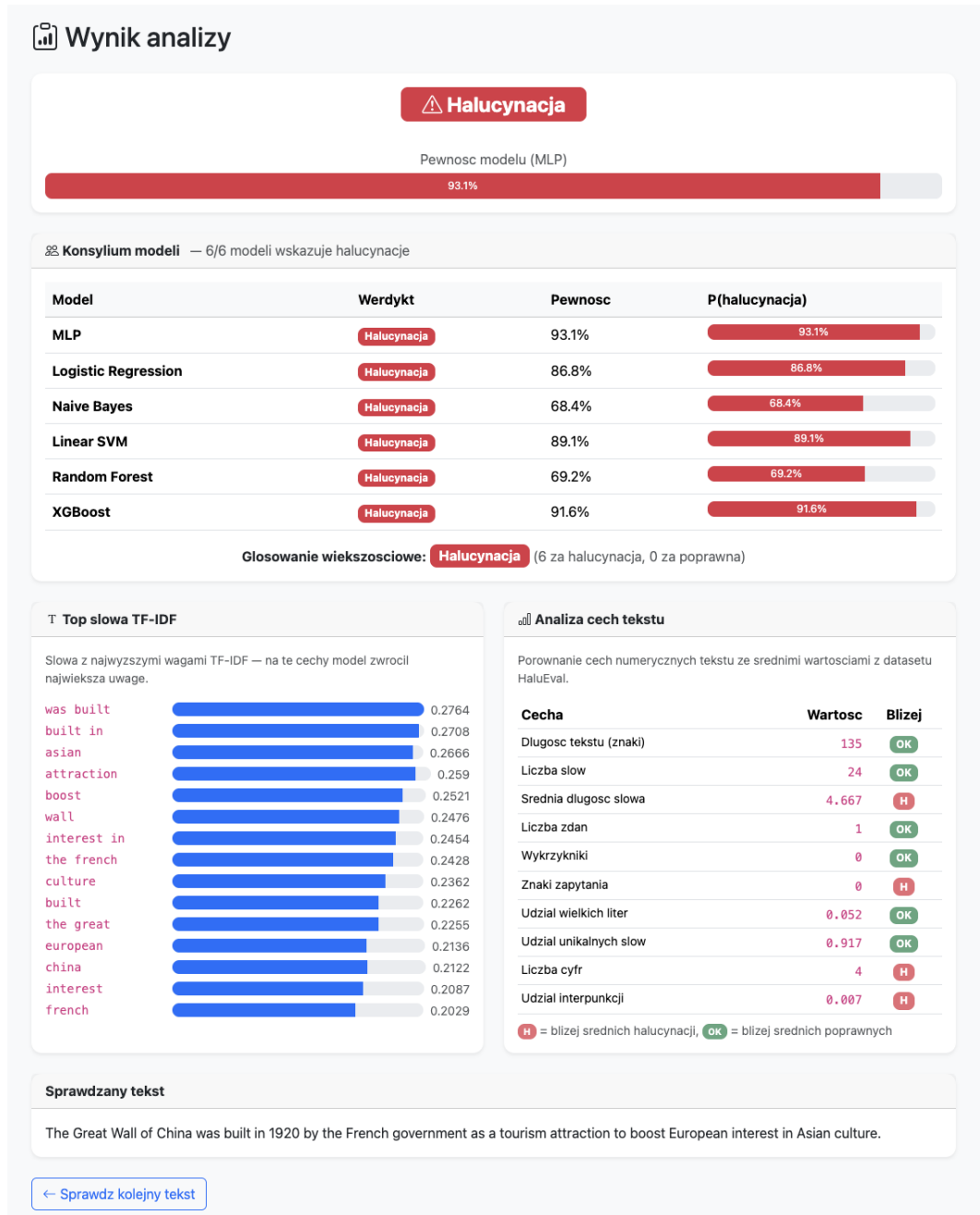
Zamiast polegać na jednym modelu, aplikacja uruchamia wszystkie 6 modeli z Project03 na tym samym tekście. Każdy model oddaje głos: halucynacja lub poprawna odpowiedź. Wynik końcowy to głosowanie większościowe.

Model	Typ	Charakterystyka
MLP	Sieć neuronowa	Warstwy (256, 128), ReLU, Adam — główny model (najwyższe F1)
Logistic Regression	Liniowy	Baseline, regularyzacja L2
Naive Bayes	Probabilistyczny	Szybki, zakłada niezależność cech
Linear SVM	Liniowy, max-margin	Kalibracja CV=3 dla <code>predict_proba</code>
Random Forest	Ensemble (bagging)	200 drzew, wysoki recall
XGBoost	Ensemble (boosting)	200 drzew, lr=0.1, drugi najlepszy model

Uwaga implementacyjna: Naive Bayes (`MultinomialNB`) wymaga nieujemnych wartości — w kodzie wartości ujemne są zerowane przed predykcją.

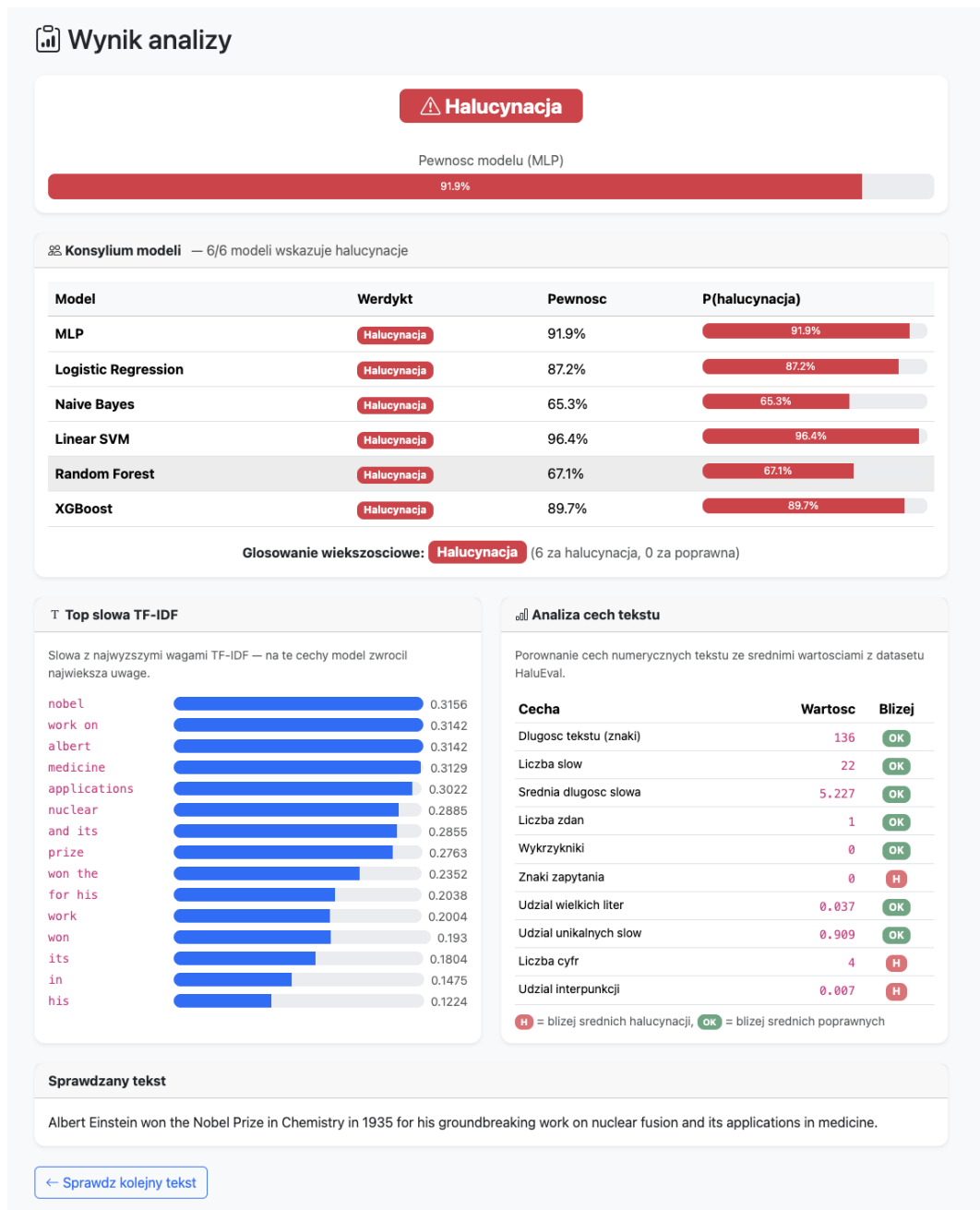
7 Przykładowe wyniki

7.1 Wykrycie halucynacji — Great Wall of China



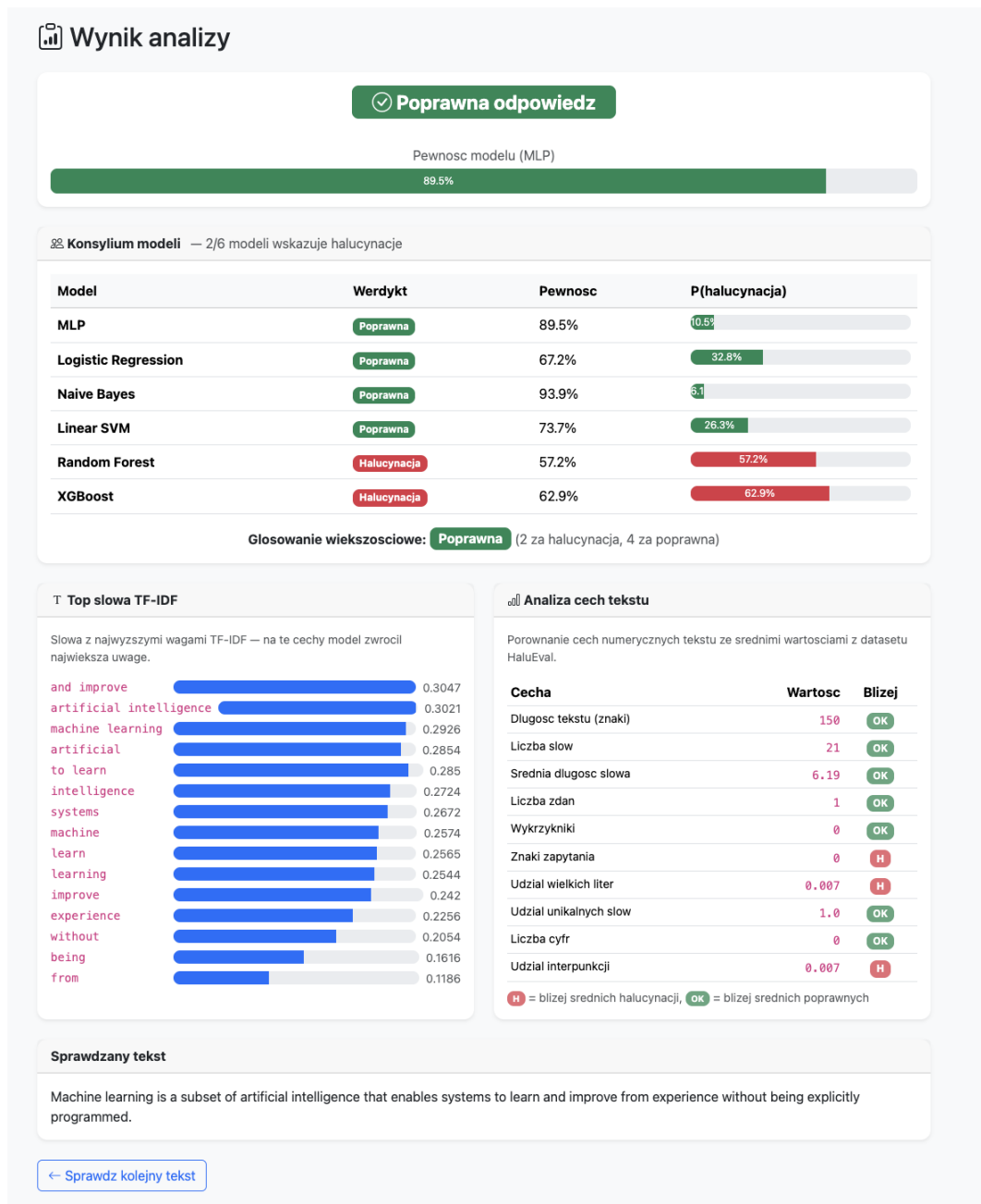
Rysunek 1: Tekst: „The Great Wall of China was built in 1920 by the French government...” — 6/6 modeli zgodnie wykrywa halucynację. MLP: 93,1% pewności. Top słowa TF-IDF: „was built”, „built in”, „asian”, „attraction”.

7.2 Wykrycie halucynacji — Albert Einstein



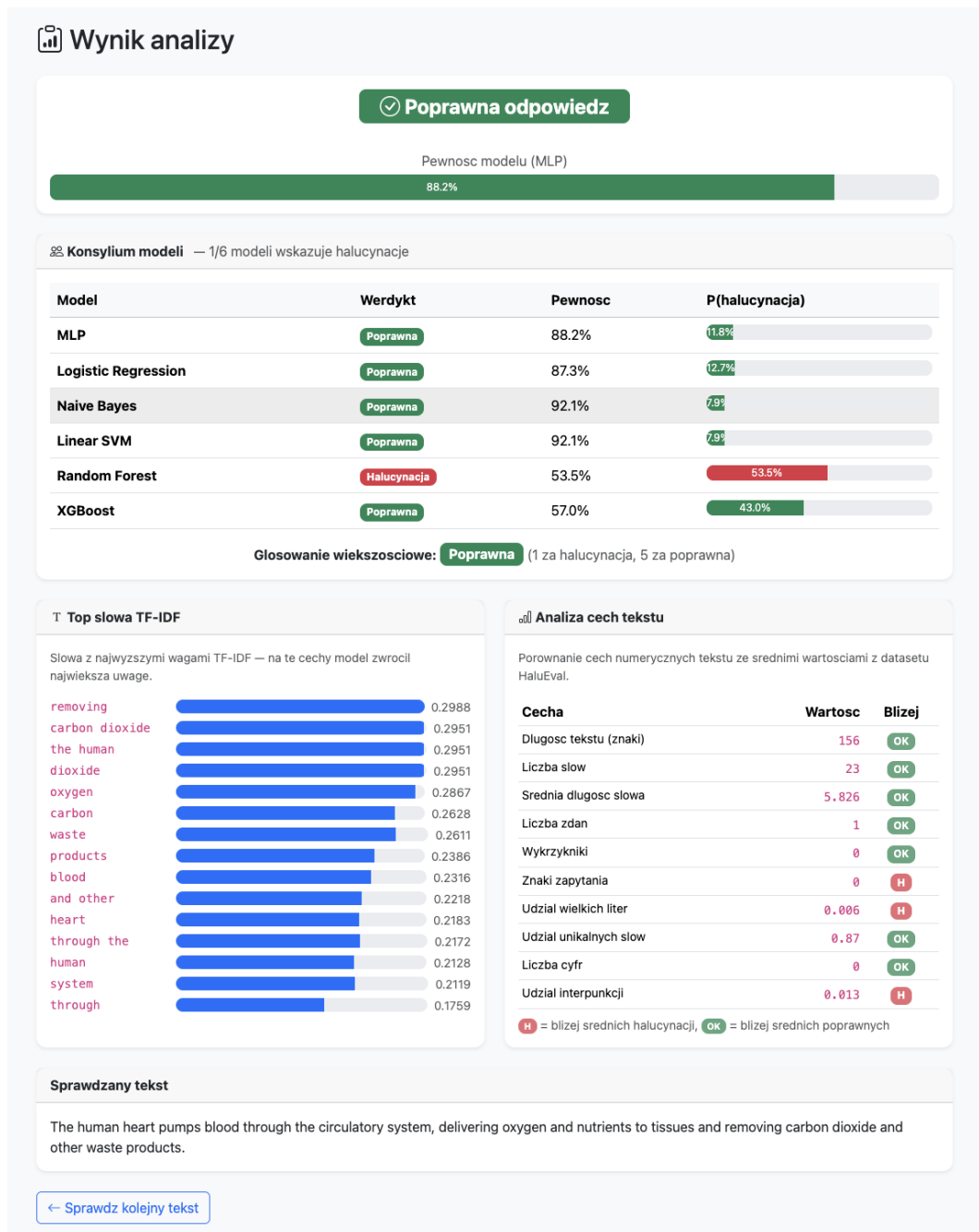
Rysunek 2: Tekst: „Albert Einstein won the Nobel Prize in Chemistry in 1935...” — 6/6 modeli zgodnie wykrywa halucynację. MLP: 91,9% pewności. Top słowa TF-IDF: „nobel”, „work on”, „albert”, „medicine”.

7.3 Poprawna odpowiedź — Machine Learning



Rysunek 3: Tekst: „Machine learning is a subset of artificial intelligence...” — 4/6 modeli głosuje za poprawną odpowiedzią. Random Forest i XGBoost głosują za halucynacją z niską pewnością (57%, 63%). MLP: 89,5% pewności za poprawną.

7.4 Poprawna odpowiedź — Human Heart



Rysunek 4: Tekst: „The human heart pumps blood through the circulatory system...” — 5/6 modeli głosuje za poprawną odpowiedzią. Jedynie Random Forest (53,5%) głosuje za halucynacją. MLP: 88,2% pewności za poprawną. Analiza cech: większość cech bliżej średnich poprawnych odpowiedzi.

7.5 Wnioski z testów

- Halucynacje faktograficzne (błędne daty, osoby, miejsca) są wykrywane z wysoką zgodnością modeli (6/6) i wysoką pewnością (>90%)
- Poprawne odpowiedzi są rozpoznawane z większą niepewnością — Random Forest i XGBoost częściej dają false positives
- Analiza cech tekstu i top słów TF-IDF dają użytkownikom dodatkowy wgląd w uzasadnienie predykcji
- Model działa najlepiej na tekstach w języku angielskim z domeny QA/streszczeń (zgodnie z danymi treningowymi HaluEval)

8 Testy

Projekt zawiera 21 testów pytest pogrupowanych w 5 kategorii:

Kategoria	Liczba	Co testuje
Testy stron	6	GET /, /about, /history — status 200, zawartość
Testy predykcji	3	POST /predict — wynik, label, pewność
Testy walidacji	3	Pusty tekst, za krótki tekst, akceptacja długiego
Testy historii	2	Zapis wpisu, czyszczenie historii
Testy rozszerzonych funkcji	7	Konsylium modeli, analiza cech, top TF-IDF
Razem	21	Wszystkie PASSED

Uruchomienie:

```
python -m pytest tests/ -v
```

9 Uruchomienie

```
# 1. Klonowanie repozytorium
git clone <url-repozytorium>
cd project04-haluccheck

# 2. Srodowisko wirtualne
python3.13 -m venv venv
source venv/bin/activate          # Linux/Mac
# venv\Scripts\activate          # Windows

# 3. Instalacja zaleznosci
pip install -r requirements.txt

# 4. Uruchomienie
python run.py
```

Aplikacja dostępna pod adresem: <http://127.0.0.1:5000>

10 Powiązanie z wcześniejszymi projektami

Element	Źródło	Użycie w HaluCheck
6 modeli ML (.pkl)	Project03 — detekcja halucynacji	Ładowane przez <code>model_loader.py</code> , używane w konsylium
TfidfVectorizer (.pkl)	Project03 — feature extraction	Transformacja nowego tekstu do przestrzeni cech
Pipeline preprocessingu	Project03 — <code>preprocessing.py</code> , <code>features.py</code>	Odtworzony w <code>model_loader.py</code> (<code>clean_text</code> , <code>extract_numeric_features</code>)
Aplikacja Flask	Doświadczenie z AstroApp i „Co mam w lodówce?”	App factory, blueprinty, szablony Jinja2

11 Podział pracy

11.1 Praca samodzielna

- Koncepcja projektu — wybór sposobu powiązania projektów
- Modele ML — wytrenowane samodzielnie w Project03
- Pipeline preprocessingu i feature engineeringu — samodzielny (Project03)
- Testowanie aplikacji, weryfikacja wyników, interpretacja predykcji
- Dokumentacja (README, niniejszy dokument)
- Integracja modeli ML z aplikacją webową Flask
- Moduł `model_loader.py` — implementacja pipeline predykcji
- Endpointy Flask (`routes.py`)
- Testy pytest (`test_app.py`)

11.2 Z pomocą AI

- Wsparcie przy pisaniu kodu — generowanie szkieletu modułów
- Debugowanie błędów (ścieżki plików, kompatybilność seaborn)
- Pomoc przy strukturyzacji projektu i planowaniu commitów

12 Bibliografia

12.1 Sztuczna inteligencja

Narzędzie	Model	Firma	Zastosowanie	Dostęp
Claude	Claude Opus 4	Anthropic	Wsparcie przy kodowaniu, struktura projektu	VI 2026

12.2 Dataset

Li, J. et al. (2023). *HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models*. EMNLP 2023. <https://github.com/RUCAIBox/HaluEval>. Dostęp: V 2025.

Dataset: <https://huggingface.co/datasets/pminervini/HaluEval> (licencja Apache 2.0). Dostęp: V 2025.

12.3 Biblioteki i frameworki

Nazwa	Wersja	Strona	Dostęp
Flask	3.1.1	https://flask.palletsprojects.com/	VI 2026
scikit-learn	1.8.0	https://scikit-learn.org/	VI 2026
scipy	1.15.2	https://scipy.org/	VI 2026
numpy	2.2.6	https://numpy.org/	VI 2026
Bootstrap	5.3.3	https://getbootstrap.com/	VI 2026
Bootstrap Icons	1.11.3	https://icons.getbootstrap.com/	VI 2026
pytest	8.3.5	https://docs.pytest.org/	VI 2026

12.4 Źródła internetowe

Tytuł	URL	Dostęp
Flask — Quickstart	https://flask.palletsprojects.com/en/stable/quickstart/	VI 2026
Flask — Application Factories	https://flask.palletsprojects.com/en/stable/patterns/appfactories/	VI 2026
Flask — Blueprints	https://flask.palletsprojects.com/en/stable/blueprints/	VI 2026
scikit-learn — Model Persistence	https://scikit-learn.org/stable/model_persistence.html	VI 2026

Tytuł	URL	Dostęp
scikit-learn — MLPClassifier	https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html	VI 2026
Bootstrap 5 — Getting Started	https://getbootstrap.com/docs/5.3/getting-started/introduction/	VI 2026
pytest — Getting Started	https://docs.pytest.org/en/stable/getting-started.html	VI 2026